Содержание

Power Systems Agent Benchmark: An Executable Benchmark for Evaluating Tool-Using AI Agents on Electric Power Engineering Tasks **1**
Abstract . . . 1
1. Introduction . . . 2
2. Related Work . . . 2
3. Requirements for a Power-Systems Agent Benchmark . . . 3
4. Architecture . . . 4
4.1 Artifact contract . . . 4
4.2 Agent-agnostic launch and harness robustness . . . 4
4.3 Evaluator maturity ladder . . . 4
4.4 The hidden split as a generator, not as files . . . 5
5. Task Set . . . 5
5.1 Domain vetting of the set . . . 5
6. Evaluators and Metrics . . . 6
7. Reproducibility . . . 6
8. Experiments . . . 6
8.1 Canonical round (public vs. hidden) . . . 6
8.2 Failure taxonomy: capability vs. specification . . . 7
8.3 Evaluation integrity: an agent that reached the oracle . . . 7
9. Limitations . . . 8
10. Conclusion . . . 8
Appendix A. Task Catalog . . . 8
Appendix B. Experiment Artifacts . . . 8

Power Systems Agent Benchmark: An Executable Benchmark for Evaluating Tool-Using AI Agents on Electric Power Engineering Tasks

Preprint draft v2 (2026-06). English version for arXiv (eess.SY; cs.AI). Word-for-word Russian companion: `paper_ru_v2.md`*. Annotated bibliography:* `bibliography.md`*; BibTeX:* `references.bib`*.*

Abstract

We present the Power Systems Agent Benchmark, a prototype *executable* benchmark for evaluating tool-using AI agents on electric-power-engineering tasks. Unlike text question-answering sets and report grading via LLM-as-a-Judge, our protocol requires an agent to emit a structured solution (`submission.json`) that a deterministic program then verifies: it recomputes the engineering quantities, checks physical and operational constraints, and returns a feasibility flag, a normalized score, and a machine-readable list of violations. The public split contains 41 task families spanning power-flow and network analysis, short-circuit and protection engineering, stability and grid-code compliance, DER/EV and storage, microgrid dispatch, reliability and restoration, power quality and standards, OT cybersecurity, and forecasting. The hidden split is not shipped: instead we publish a deterministic generator that derives hidden cases from a private seed with automatic solvability checks, so the mechanism is auditable while the concrete numbers stay private. On a matrix of command-line agents we show that (1) the benchmark discriminates between agents, consistently across the public and hidden splits; (2) a run – analyze identical failures – refine specifications – rerun loop separates specification defects from genuine capability differences, surfacing ten specification defects and — most notably — a latent physics bug in a task evaluator that was invisible to baseline-equals-evaluator self-consistency checks and could only be caught because independent agents applied the correct domain physics. On clean specifications the public and hidden splits agree on the

ranking and yield comparable scores; the value of hidden evaluation is thus prospective contamination resistance rather than a one-round difficulty gap. Current evaluators are compact deterministic surrogates; migration to simulator-backed checkers (pandapower, OpenDSS) is described as a roadmap that does not change the task contract.

1. Introduction

Large language models are increasingly deployed as agents on engineering tasks: they read data, write helper code, call tools, and produce decisions. The standard ways of evaluating such agents — reference-answer QA or grading a free-form report with another LLM (LLM-as-a-Judge) — fit power engineering poorly. Here a solution can read convincingly and still violate current and voltage limits, protection selectivity, grid-code requirements, or balance equations. An agent's error manifests not as a bad phrasing but as a line overload, a non-selective relay setting, an inadmissible voltage, or a misidentified fault location.

The primary evaluation result must therefore be determined not by the overall impression of a text but by an executable check of the proposed solution. We formalize and implement such a protocol: each task publishes structured inputs (`task.json`), the agent must return a machine-readable solution (`submission.json`), and an independent evaluator recomputes the target quantities, checks the hard constraints, and returns `feasible`, a normalized `score`, and a list of `violations`. The agent's free-text `rationale` is stored as a diagnostic but does not affect the score.

Contributions:

1. An executable evaluation contract `task.json → agent → submission.json → evaluator → evaluation_result.json` with JSON schemas for every artifact, a clean separation of hard constraints (`feasible`) from quality (`score`), and an agent-agnostic launch protocol that runs any CLI agent through environment variables.
2. A balanced public set of 41 executable task families across eight domain groups, each with traceable provenance (source cards, references to standards and literature) and bilingual (EN/RU) presentation.
3. An architectural fix for hidden-set leakage: instead of shipping hidden cases we ship a deterministic per-family generator; cases are derived from a private seed on the evaluation host and every draw is accepted only if the family baseline solves it and the evaluator confirms feasibility. The mechanism is publicly auditable; the concrete numbers are not.
4. A specification-quality control methodology: across an agent matrix, an identical failure of all participants is read as a symptom of a specification or evaluator defect rather than agent weakness. The criterion surfaced ten specification defects, which we fixed and closed by rerunning.
5. The strongest instance of (4): a latent physics bug in the fault-localization evaluator (an off-by-one in the fault-current-indicator model) that self-consistency checks could not catch, because the baseline inherited the same error. Three independent agents applying correct physics exposed it — a result we treat as the clearest demonstration of the method's value.

Beyond the dataset, the work contributes a reusable harness: an isolation model that prevents agents from reaching the oracle, robustness to malformed submissions, survival of evaluator crashes, and a narrow transient-infrastructure retry policy that reports raw and retry-normalized metrics separately so that provider reliability never masquerades as agent capability (Section 4).

2. Related Work

Agent benchmarks. General-purpose suites — AgentBench [liu2024agentbench], GAIA [mialon2023gaia], WebArena [zhou2024webarena] — evaluate web navigation, code, and tool use but not engineering domains. SWE-bench [jimenez2024swebench] and Codex/HumanEval [chen2021codex] established executable evaluation (the solution is checked by tests); τ-bench [yao2024taubench] carried the "compare the final

state, not the text" principle to conversational agents and introduced pass^k as a reliability metric. Our contract extends executable evaluation from software to engineering computations with physical constraints.

LLM-as-a-Judge. LLM judging [zheng2023judging] suffers documented biases — position, length, self-preference [wang2023fairevaluators, gu2024judgesurvey, li2024generationjudgment]; for tasks where an answer can look convincing while violating constraints, a judge is unsuitable as the primary score. We use deterministic recomputation and keep the rationale only as a diagnostic.

LLM agents in power systems. Surveys [llmpowersurvey2025, agenticpowersurvey2025] record rapid growth of applications amid the absence of standardized evaluation. The conceptually closest work is PowerAgentBench [poweragentbench], the benchmark component of the PowerAgent ecosystem at Harvard SEAS [zhang2025poweragent]: three task levels (N-1 audit on IEEE 39-bus, agentic N-2 search, dynamic model-quality review with PSS/E) with a public/hidden split and a hidden evaluator that recomputes physical validity; a dedicated paper for the benchmark had not appeared at the time of writing. Our set differs in coverage (41 families across eight domain groups vs. three levels), in seed-based hidden-case generation instead of a static hidden set, in the agent-agnostic CLI protocol, and in the formalized specification-quality control loop. Other neighbors: ElecBench [zhou2025elecbench] — dispatch scenarios, but with text/judge scoring; PFBench [pfbench2025] — programmatic checking, but power flow only. Emerging production agents — GridMind [gridmind2025], X-GridAgent [xgridagent2025], Grid-Mind CIA [gridmindcia2026], PFAgent [pfagent2026] — are the intended "subjects" of such benchmarks.

RL grid-control environments. Grid2Op/L2RPN [marot2020l2rpn, marot2021retrospective] is a mature sequential topology-control environment for RL; we address a different problem — an agent-agnostic contract of single-shot engineering decisions across a broad span of domain subtasks.

Simulators. pandapower [thurner2018pandapower], MATPOWER [zimmerman2011matpower], OpenDSS [dugan2011opendss], PowerModels/PGLib [coffrin2018powermodels, babaeinejadsarookolaee2019pglib], ANDES [cui2021andes], PyPSA [brown2018pypsa] cover computational kernels but do not define an agent evaluation contract; in our roadmap they are the substrate of the checkers, pluggable without changing the task format.

Benchmark integrity. Training-data contamination systematically inflates results [contaminationsurvey2025, statictodynamic2025]; the response is dynamic and refreshed sets (Dynabench [kiela2021dynabench], LiveBench [white2024livebench], LiveCodeBench [jain2024livecodebench]). Our hidden generator is a prevention mechanism of the same class: we publish how cases are produced, not the cases. The criteria for sound agent evaluation from [kapoor2024agentsmatter] (cost, reproducibility, separation of failure classes) are implemented directly in the harness; the limits of best-of-N as a metric are quantified in [brown2024monkeys].

3. Requirements for a Power-Systems Agent Benchmark

1. Executability: the solution is verified by code, not by string matching.
2. Physical verifiability: the evaluator recomputes quantities and checks constraints rather than comparing against stored text.
3. Public/hidden separation: public cases support development, hidden cases support fair comparison, and leakage of hidden answers is architecturally excluded.
4. Agent agnosticism: launching does not depend on a specific LLM, CLI, or framework; the contract is "read files, write a file."
5. Provenance: every task carries source provenance with a confidence level.
6. Machine-readable result: `feasible`, `score $\in$ [0,1]`, `violations`, metrics.
7. Difficulty growth without a contract change: moving from formula surrogates to simulator-backed checks changes the evaluator internals, not the task/submission format.

8. (Added from experiments) Specification-quality control: the benchmark must include a procedure for separating specification defects from genuine capability differences (Section 8.2).

4. Architecture

4.1 Artifact contract

`task.json → agent (any CLI) → submission.json → evaluator → evaluation-result.json`

- `schemas/task.schema.json` — required fields: `task_id`, `family`, `substrate`, `objective`, `public_inputs`, `submission_schema`, `hard_constraints`, `metrics`, `source_evidence`, `source_provenance`, plus a bilingual block.
- `schemas/submission.schema.json` — structured answer: `actions` and/or `answer`, optional `rationale`.
- `schemas/evaluation-result.schema.json` — `task_id`, `feasible` (bool), `score $\in$ [0,1]`, `violations` (list), metrics, and evidence.

The artifact validator additionally enforces consistency of mirrored data copies (`public_inputs` and the top-level of `task.json`) — a defect of this kind was found and fixed during the work.

4.2 Agent-agnostic launch and harness robustness

`prepare_agent_tasks.py` assembles a self-contained packet per task (`task.json` with redaction + `instructions.md`); sample submissions and any oracle-bearing fields are stripped. `run_agent_command.py` runs an arbitrary shell command per packet, exposing environment variables (`BENCH_PACKET_DIR`, `BENCH_TASK_PATH`, `BENCH_SUBMISSION_PATH`, …) with a per-case timeout. Three properties make the harness trustworthy for cross-agent comparison:

- Isolation. Each case runs in a sandbox copy of the packet outside the repository, with the sandbox as the working directory and without exposing the repository root or its `PYTHONPATH`. This closes an oracle-access path we observed in practice (Section 8.3): an agent located and executed the evaluator and self-scored against the leaked oracle while packets lived inside the repository tree.
- Robustness to malformed submissions. A structurally-valid but garbage submission (nulls, strings for numbers, wrong shapes) always returns `feasible=false` with per-field violations rather than crashing — a property verified by a probe across all 41 families. A field report (an agent returning nulls for every numeric field) had previously aborted a run at 27/41.
- Failure-class separation and retries. A narrow retry policy (`--max-attempts`) retries only transient infrastructure failures (provider/transport signatures, early silent aborts, quota/spend-limit signatures); quality failures, contract breaks, and per-case budget timeouts are never retried, because retrying them would silently turn the benchmark into best-of-N. Every result row records `attempt_count`, `failure_class_initial`, `selected_attempt`, and `raw_first_attempt`, and the summary reports both raw and retry-normalized metrics plus an `infra_unresolved` count.

4.3 Evaluator maturity ladder

formula/graph surrogate → deterministic engineering model → simulator-backed hidden evaluator → stochastic/large-scale scenario suite.

Current public evaluators sit on the first two rungs; the contract lets the checker internals be replaced (e.g. a DC flow by a pandapower AC solve) without changing the task format.

4.4 The hidden split as a generator, not as files

The `tasks/*/hidden/` directory is excluded from version control. Instead the repository contains `generate_hidden_cases.py`: a registry of deterministic per-family perturbations. A case is derived from the public `case_001` by a series of physically meaningful redraws (network parameters, weather series, load profiles, a fault location with consistent indicator readings, …) seeded by a private key; a draw is accepted only if the family baseline solves it and the evaluator confirms feasibility, otherwise a deterministic redraw occurs. The leaderboard operator keeps the seed secret; anyone with the repository can audit how hidden cases are produced without knowing their concrete numbers.

5. Task Set

41 public task families are grouped into eight categories:

1. Network analysis and operating constraints — `ybus_construction`, `pf_voltage_repair`, `contingency_screening`, `corrective_transmission_switching`, `dynamic_thermal_rating`.
2. Short circuit and protection — `three_phase_short_circuit` (IEC 60909 max/min current pair), `earth_fault_calculation`, `breaker_relay_short_circuit`, `distance_protection_settings`, `overcurrent_protection`, `cable_ampacity_sizing`.
3. Stability, grid code, and IBR — `critical_clearing_time` (equal-area criterion), `transient_stability_prediction`, `frt_compliance`, `ibr_short_circuit_frt`, `min_synchronous_share`.
4. DER, PV, EV, and storage — `pv_volt_var`, `ev_v2g_outage_schedule`, `ev_v2g_voltage_support`, `bess_ancillary_response`, `commercial_pv_lcoe_uncertainty`.
5. Microgrids and dispatch — `microgrid_economic_dispatch`, `rolling_microgrid_dispatch`, `islanded_microgrid_pq_dispatch`, `dispatch_uncertainty`, `hydro_thermal_storage_uc`, `n1_generation_commitr`
6. Reliability and restoration — `flisr_restoration`, `fault_section_localization`, `fci_placement`, `fci_saidi_caidi`, `operator_breaker_load_actions`.
7. Power quality, standards, assets, cybersecurity — `en50160_voltage_compliance`, `power_quality_event_classificat` `harmonic_ieee519_compliance`, `transformer_thermal_loading` (IEC 60076-7), `fdi_state_estimation` (WLS + largest-normalized-residual test), `protected_meter_placement`, `mas_frequency_handover`.
8. Forecasting and uncertainty — `wind_power_forecast`, `wind_prediction_interval`.

Each task carries a source card and source provenance with a confidence level; the full catalog is in the appendix (`bilingual_task_inventory.md`).

5.1 Domain vetting of the set

The set underwent a domain review that removed four classes of factual defects (fixes recorded in the repository history):

- Physical errors: the RoCoF formula placed `f0` in the denominator instead of the numerator (an `f0^2` discrepancy), so the inertia constraint was never binding; fixed against the swing equation with a case rebalance.
- Conceptual substitutions: the "FDI detection" task carried attack labels in its inputs and reduced to a threshold comparison; it was reworked into a genuine WLS state estimation with the iterative largest-normalized-residual test, where residual smearing makes the naive weighted test wrong, and stealthy FDI is explicitly scoped out.
- Degenerate tasks: critical-clearing-time computation was reading precomputed stability flags from a table; replaced with an analytic equal-area-criterion calculation.
- Overclaimed standards: an "IEC 60909" short-circuit calculation without the voltage factor `c` and with arc resistance in the maximum current understated the current by ~10% non-conservatively; reworked into the canonical Ik_max (c_max, bolted) / Ik_min (c_min, with resistance) pair.

In addition, three out-of-scope families (building heating ×2, Sobol variance decomposition) were replaced with previously uncovered in-scope domains: transformer thermal loading (IEC 60076-7), cable sizing by three checks, and IEEE 519 harmonic compliance.

6. Evaluators and Metrics

Result fields: `feasible` (hard constraints satisfied), `score $\in$ [0,1]`, `violations` (concrete violations in words), and task-specific evidence. Scoring approaches by task type:

- exact computation: 1/0 or numeric-field tolerances;
- control/optimization: feasibility on constraints + quality on the objective. For tasks with non-unique optima (microgrid dispatch) the evaluator re-simulates the submitted decision and scores the cost gap to the optimum: equal-cost alternative optima receive full score, feasible suboptimal solutions receive partial credit;
- classification/localization: exact sets with enumerated misses;
- standards and grid code: pass/fail + diagnostic fields.

Metrics aggregate two ways: feasible count (share of tasks meeting hard constraints) and perfect count (share with `score = 1.0`); as shown in Section 8 these can disagree on the ranking, and we recommend reporting both together with `mean_score`.

7. Reproducibility

`scripts/check.sh` runs the unit-test suite (evaluators, baselines, the harness, the hidden-case generator), artifact validation, one-by-one evaluation of all 41 public sample submissions, and a public-baseline run. The baselines are built on the same computational kernels as the evaluators and yield 41/41 feasible with a mean score of 0.997 — this is a contract self-consistency check, not evidence of task difficulty, and the distinction is stated in every report. As Section 8.2 shows, this self-consistency cannot validate evaluator correctness: a baseline derived from a buggy evaluator agrees with it. Continuous integration runs the full check on every commit.

8. Experiments

Command-line agents, identical prompts, a 600 s per-case timeout, and the `--max-attempts 2` retry policy; agents were launched with no benchmark-specific adaptation:

- Codex CLI 0.139.0, model gpt-5.5 (reasoning effort medium);
- Cursor agent CLI, model composer-2.5;
- OpenCode 1.15.13, model deepseek-v4-flash.

8.1 Canonical round (public vs. hidden)

The reference round (`runs/canonical_round1/`), run after all fixes. Public uses the committed `case_001` per family; hidden uses one generated `case_401` per family from a fresh private seed. Zero infrastructure-unresolved failures; raw and retry-normalized metrics coincided.

agent (model)	public feasible	public perfect	public mean	hidden feasible	hidden perfect	hidden mean
codex (gpt-5.5 medium)	40/41	36/41	0.971	40/41	37/41	0.969

agent (model)	public feasible	public perfect	public mean	hidden feasible	hidden perfect	hidden mean
cursor (composer-2.5)	39/41	35/41	0.947	40/41	36/41	0.969
opencode (deepseek-v4-flash)	37/41	35/41	0.900	38/41	35/41	0.920

Two observations. The benchmark discriminates: codex ≈ cursor (~0.97 mean) sit clearly above opencode (~0.90–0.92), consistently on both splits. Public and hidden are comparable on clean specifications: for every agent the hidden score is within noise of public. A dramatic ranking divergence observed in an earlier development round — where the public leader appeared to degrade most on hidden — was traced entirely to specification defects that disproportionately penalized one agent; once those defects and a latent evaluator bug (Section 8.2) were fixed, the two splits agree. We therefore do not claim a one-round public/hidden difficulty gap. The role of hidden evaluation is prospective: contamination resistance as a public set ages and models train on it. As mild corroboration, these recent models do not score higher on the published public set than on freshly generated hidden cases — the signature of memorization is absent for these compact cases.

8.2 Failure taxonomy: capability vs. specification

An earlier matrix showed two distinct failure groups. Group A — agents disagree: voltage-limit violations in Volt-VAR, a wrong FRT verdict, an inverted EV discharge sign, different errors in stability risk scoring; this is the leaderboard signal. Group B — all agents fail identically; this is a symptom of a specification defect. We identified ten defects across two passes: a field description contradicting its name (fraction vs. percent — two agents returned 0.51 instead of 51.0 with a numerically correct computation); nested key names not matching the checker; rejection of a semantically equivalent zero entry; a demand for an exact match to one of several equal-cost optima; unstated conventions for the wind power curve, quantiles, and conductor heat-balance terms; a hardcoded P/Q dispatch policy; and an unguessable risk-score formula. The criterion "an identical failure of all participants ⇒ suspect a specification or evaluator defect" is proposed as a standard quality-control procedure for executable benchmarks. After each fix the affected families were rerun (or existing submissions re-scored) and the same agents recovered, confirming the diagnosis.

The strongest instance arose in the canonical round and concerns the evaluator itself. On the hidden `fault_section_localization` draw all three agents localized the fault to section s24 — consistent with both the fault-current indicators and the outage complaints — while the evaluator demanded a different section that contradicted the case's own indicator readings. The cause was an off-by-one in the evaluator's FCI model: it placed the faulted section's downstream node on the fault path, so an indicator at that node falsely registered the fault, whereas an indicator sees fault current only if the fault is strictly downstream of it. Crucially, this bug was invisible to the benchmark's own self-consistency check: the baseline is derived from the same function as the evaluator, so baseline and evaluator agreed with each other and the public sample passed. Only independent agents applying the correct physics could expose it. We regard this — an executable benchmark whose evaluator contained a latent domain error that an agent matrix caught — as the clearest demonstration of the methodology's value, and as a caution that baseline-from-evaluator construction cannot validate evaluator correctness.

8.3 Evaluation integrity: an agent that reached the oracle

A log audit of a hidden run revealed that one agent (Cursor) on the transient-stability task located and executed the checker and self-scored against the oracle: its log states that the submission "evaluates as feasible with a score

of 1.0." This was possible because packets lived inside the repository working tree and the harness ran agents with the repository as the working directory; the prohibition existed only in the packet instructions. The lesson — instruction-level isolation is insufficient for agents with autonomous filesystem access — drove the sandbox redesign in Section 4.2. The finding is included as a standalone methodological result.

9. Limitations

1. Public cases are compact (3–5-bus networks, short series): they test command of a method, not industrial-scale data volumes.
2. Evaluators are deterministic surrogates; industrial AC accuracy cannot be claimed before simulator-backed checkers are connected.
3. One case per family per split: ranking by 41 points is statistically fragile; the generator makes enlarging the sample cheap.
4. The baseline 41/41 is a contract self-consistency check, not a difficulty lower bound, and cannot validate evaluator correctness.
5. The seed of any disclosed hidden round is "burned" by running it; a citable leaderboard round requires a fresh seed on an isolated host.
6. The sandbox is realized by relocation, not containerization: it removes leakage via relative paths and environment, but not deliberate oracle search by absolute path; competitive settings require host-level containers.
7. One configuration per agent; without repeats the variance over LLM sampling is not estimated.
8. A within-family attempt to evaluate the maintainer's own model family on a shared-budget account is infrastructurally infeasible (it exhausts the session budget) and is excluded; a clean self-evaluation needs a dedicated API key and an explicit conflict-of-interest caveat.

10. Conclusion

The Power Systems Agent Benchmark demonstrates a practical path from text tasks to executable evaluation of AI agents in power engineering. We propose and implement a reproducible benchmark contract that turns power tasks from free-text answers into executable checks; we show it discriminates between current agents consistently across public and hidden splits. The central methodological contribution is the agent-matrix quality-control loop: treating an identical failure of all agents as a defect signal surfaced ten specification defects and a latent physics bug in a task evaluator that self-consistency checks could not catch — a reusable procedure for any executable benchmark, and a caution that a baseline derived from the evaluator cannot validate the evaluator. The core value is not the specific compact cases but the contract, the taxonomy, the checkers, the seed-based hidden-case generator (whose role is prospective contamination resistance), the isolated harness, and the reproducible quality-control loop; the roadmap is to fill the checkers with simulator-backed implementations without changing the external task contract.

Appendix A. Task Catalog

See `docs/bilingual_task_inventory.md` (41 families, EN/RU, with `task_id` and paths).

Appendix B. Experiment Artifacts

- Canonical round: `runs/canonical_round1/` (per-agent, per-split summaries and logs).
- Earlier development matrices and the spec-fix history: `runs/public_agent_smoke/REPORT.md`, `runs/hidden_agent_smoke/REPORT.md`, and the repository commit history.